

# **Complete DSA Learning Order**

Topic Content Compilation

Section 6 — Binary Search (6.1 – 6.4)

June 30, 2026

## Section 6 — Binary Search

### 6.1 — Classic binary search on a sorted array

#### 1. PLAIN-LANGUAGE GIST

Binary search finds a target value in a sorted array by repeatedly checking the middle element and discarding the half of the array that can't possibly contain the target — instead of checking every element one by one, you eliminate half the remaining possibilities with every single comparison.

#### 2. REAL-WORLD ANALOGY

Think of looking up a word in a physical dictionary. You don't start at page 1 and flip forward one page at a time — you open to roughly the middle, see whether your word comes before or after that page alphabetically, and then repeat that same “jump to the middle of what's left” process on just the relevant half, narrowing in rapidly rather than scanning sequentially.

#### 3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, concrete instance of 0.2's  $O(\log n)$  signature — the “loop where the increment is multiplicative” pattern from 0.3 (where each step halves the remaining problem size) is exactly what binary search's left/right shrinking does, just applied to a search range rather than a simple counter.

#### 4. CORE MECHANISM (Simple → Intermediate)

**Simple model:** Maintain two pointers, left and right, marking the current range still worth searching. Compute mid (the midpoint of that range). If `arr[mid]` equals the target, you're done. If `arr[mid]` is too small, the target (if present) must be to the right, so discard the left half by moving left past mid. If `arr[mid]` is too large, discard the right half by moving right before mid. Repeat until found or the range becomes empty.

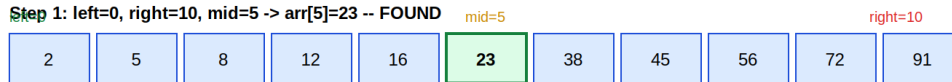
**Next layer — why this requires sortedness, and why it's  $O(\log n)$ :**

- The entire technique depends on one guarantee: if `arr[mid]` is smaller than the target, every element to the left of mid is also smaller (because the array is sorted) — so the entire left half can be safely discarded without checking any of it individually. Without sortedness, this elimination would be unjustified, and you could be discarding the very element you're looking for.
- Each comparison cuts the remaining search space exactly in half, which is precisely the doubling-in-reverse pattern from 0.3's  $i *= 2$  example — starting from  $n$  elements, after  $k$  halvings only  $n / 2^k$  remain, and solving for when that shrinks to 1 gives  $k = \log_2(n)$ , the formal reason binary search is  $O(\log n)$ .
- This is the practical payoff of 1.2's two-pointer technique combined with 0.2's complexity vocabulary — binary search is, in a real sense, a two-pointer technique where the pointers don't move one step at a time but jump straight to a calculated midpoint each round.

#### 5. VISUAL REPRESENTATION

### Binary search for 23 in [2,5,8,12,16,23,38,45,56,72,91]

Step 1: left=0, right=10, mid=5 -> arr[5]=23 -- FOUND



Suppose instead we were searching for 45 (not 23) -- trace continues:

Step 1: mid=5 -> arr[5]=23 < 45 -> target is to the RIGHT -> left = mid+1



Step 2: new range is [38,45,56,72,91] -- HALF the array eliminated in one comparison

Every comparison eliminates HALF the remaining search space -- starting from  $n$  elements, after  $k$  comparisons only  $n/2^k$  remain. Solving  $n/2^k = 1$  gives  $k = \log_2(n)$  -- this is exactly why binary search is  $O(\log n)$ .

The trace shows searching for 23: the midpoint lands directly on it in one comparison. The second trace (searching for 45) shows the more typical case — one comparison eliminates an entire half of the array, and the next midpoint is recalculated only within the surviving half.

## 6. WHEN TO USE / WHEN NOT TO USE

### Use classic binary search when:

- The data is already sorted (or sorting it first is cheap enough relative to how many searches you'll perform — recall 0.4/0.6's amortized-cost thinking: one  $O(n \log n)$  sort followed by many  $O(\log n)$  searches can be a worthwhile tradeoff).
- You need to find an exact value, or the boundary where a condition flips (the basis for 6.2 and 6.3).

### Wrong choice when:

- The data isn't sorted and won't be searched repeatedly enough to justify sorting it first — a single search on unsorted data is just  $O(n)$  regardless (1.1), and sorting purely to enable one binary search adds unnecessary  $O(n \log n)$  overhead.
- You need to find all occurrences of a value or a full traversal of matching elements rather than a single position — binary search locates a boundary or position; finding every match generally still needs to scan outward from that position or use a different technique entirely.

## 7. C# EXAMPLES — Simple → Intermediate → Advanced

### SIMPLE — classic binary search, the exact trace above:

```
int BinarySearch(int[] arr, int target) {
    int left = 0, right = arr.Length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // avoids overflow
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1; // discard left half
        else right = mid - 1; // discard right half
    }
    return -1; // not found
}
```

**INTERMEDIATE — finding the first occurrence of a target when duplicates exist (a common follow-up question):**

```

int FindFirstOccurrence(int[] arr, int target) {
    int left = 0, right = arr.Length - 1, result = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            result = mid;          // record this match...
            right = mid - 1;       // ...keep searching LEFT for earlier
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}

```

The key shift here: finding a match doesn't immediately return — it records the match and continues narrowing toward the left, since an earlier occurrence might still exist. This pattern (don't stop at the first match, keep narrowing toward a boundary) is the direct precursor to 6.2's boundary-finding generalization.

**ADVANCED — searching a 2D matrix that's sorted both row-wise and column-wise, treating it as a single flattened sorted sequence:**

```

bool SearchMatrix(int[,] matrix, int target) {
    int rows = matrix.GetLength(0), cols = matrix.GetLength(1);
    int left = 0, right = rows * cols - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int midValue = matrix[mid / cols, mid % cols]; // -> 2D coords

        if (midValue == target) return true;
        if (midValue < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}

```

The clever part: `mid / cols` and `mid % cols` convert a single flattened index back into row/column coordinates, letting the exact same one-dimensional binary search logic operate transparently over a 2D structure — recognizing that a sorted 2D grid (with each row's last element smaller than the next row's first) is really just a sorted 1D array in disguise.

## 8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, binary search generalizes far beyond “find an exact value” into searching over any monotonic predicate (formalized next in 6.2), and the C# standard library exposes this directly via `Array.BinarySearch()` for simple exact-value lookups — worth knowing it exists, though most interview contexts want you to implement the logic yourself to demonstrate understanding. There's also a subtle correctness concern worth knowing: the naive `mid = (left + right) / 2` can integer-overflow for extremely large arrays (if `left + right` exceeds `int.MaxValue`), which is why `mid = left + (right - left) / 2` is the more defensive, universally-safe formulation — rarely triggered with small interview-sized inputs, but a detail that signals real attention to correctness if mentioned proactively.

## 9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is attempting binary search on unsorted data, expecting it to “still sort of work” because the array merely looks organized — without the sortedness guarantee, the core

elimination logic (discarding an entire half based on one comparison) is simply invalid, and the algorithm can silently return wrong results rather than crashing. Another is inconsistent loop and update conditions — using  $\text{left} \leq \text{right}$  but updating bounds as if using  $\text{left} < \text{right}$  (or vice versa) is the single most frequent source of off-by-one bugs in binary search, examined in depth in 6.4. A third: assuming binary search always finds “a” match is the same as finding “the first” or “the last” match — as the intermediate example shows, finding a specific boundary among duplicates requires deliberately continuing to narrow after finding a match, not stopping immediately.

## 10. SELF-CHECK

- Why does binary search's correctness fundamentally depend on the array being sorted — what specific step in the algorithm would become invalid on unsorted data?
- Walk through, out loud, why each comparison in binary search eliminates exactly half the remaining search space, and how that leads to the  $O(\log n)$  complexity.
- In the “find first occurrence” example, why does the algorithm continue narrowing left even after finding a matching element, instead of returning immediately?

## 6.2 — Identifying when a problem is “binary searchable” even without an obviously sorted array

### 1. PLAIN-LANGUAGE GIST

Binary search doesn't actually require a sorted array — it requires a monotonic true/false predicate, meaning some yes/no question whose answer flips from true to false (or false to true) exactly once as you scan across the space, with no flipping back and forth. Recognizing this underlying requirement, separate from “is the array literally sorted,” is what lets you apply binary search to problems that don't look like classic sorted-array search at all.

### 2. REAL-WORLD ANALOGY

Picture tuning an old analog radio dial searching for the exact point where a station's signal becomes clear. As you turn the dial in one direction, the signal doesn't randomly flicker between clear and static — it's static, then static, then at some specific point it becomes clear, and stays clear from there onward (within that station's range). You don't need the entire dial to be “sorted” in any literal sense — you just need that one clean transition point, and you can find it by repeatedly checking the midpoint of your current range and narrowing toward the transition, exactly like binary search.

### 3. CONNECT TO PRIOR KNOWLEDGE

This generalizes 6.1's mechanism by replacing “is arr[mid] equal to/less than/greater than target” with any monotonic yes/no check — the halving logic is identical, only the question being asked at each midpoint changes, which is precisely the kind of abstraction-over-mechanism thinking 5.3 demonstrated when permutations and combinations turned out to share the same backtracking template.

### 4. CORE MECHANISM (Simple → Intermediate)

**Simple model:** Define a predicate function Check(position) that returns true or false. If, across the entire search space, all the true answers come before all the false answers (or vice versa) — with no mixing — you can binary search for the exact boundary where the answer flips, using the same left/right/mid shrinking logic as 6.1, just deciding which half to discard based on Check(mid) instead of a direct value comparison.

**Next layer — recognizing this property in problems that don't look sorted:**

- A rotated sorted array (sorted, then split and swapped, like [45,56,72,91,2,5,8,12,16,23,38]) isn't sorted overall, but the predicate “is this element  $\geq$  the first element” is still monotonic — true for the entire first (rotated-in) segment, false for the rest — letting you binary search for the rotation point or adapt the search to find a target despite the rotation.
- Many “find the boundary” problems are phrased as optimization rather than search — “find the first day temperature exceeds X,” “find the smallest value satisfying some condition” — and the underlying predicate (“has this condition become true yet?”) is monotonic across the relevant range even though no array is explicitly being searched at all.
- The general recognition test: ask “if I check the middle of my current range, does knowing the answer there definitively tell me which half can be safely discarded?” If yes — if the answer's truth value is consistent across each half — the problem is binary searchable, regardless of whether the underlying data structure is a literal sorted array.

### 5. VISUAL REPRESENTATION

**"Binary searchable" = a monotonic true/false predicate, not just sortedness**

Rotated array [45,56,72,91,2,5,8,12,16,23,38] -- find 91. Predicate: "is arr[mid] >= arr[0]?"

|    |    |    |    |   |   |   |    |    |    |    |
|----|----|----|----|---|---|---|----|----|----|----|
| 45 | 56 | 72 | 91 | 2 | 5 | 8 | 12 | 16 | 23 | 38 |
| T  | T  | T  | T  | F | F | F | F  | F  | F  | F  |

The predicate "arr[mid] >= arr[0]" is T,T,T,T,F,F,F,F,F,F -- a clean TRUE-then-FALSE pattern, even though the array itself isn't fully sorted -- this monotonicity is what matters.

**The real requirement for binary search:**

There must exist SOME yes/no question about each position such that the answers form an unbroken run of TRUE followed by an unbroken run of FALSE (or vice versa) across the whole search space -- sortedness is just the most common way to get this property, not the only way.

**Binary search the BOUNDARY between true and false, not literal values.**

*The diagram applies the predicate "is arr[mid] >= arr[0]?" across a rotated array: the result is a clean run of Ts followed by a clean run of Fs, despite the array overall not being sorted — this monotonic pattern is the real requirement binary search depends on, with literal sortedness being just the most common way to produce it.*

## 6. WHEN TO USE / WHEN NOT TO USE

### Recognize binary searchability when:

- A problem involves finding a specific transition point, threshold, or boundary, and you can articulate a yes/no question per candidate position that's consistent (doesn't flip back and forth) across the range.
- The problem mentions "rotated," "first/last position where," "minimum value such that," or similar boundary-oriented language — these are strong signals, even when no array sorting is mentioned directly.

### Wrong choice / traps to avoid:

- Assuming any problem involving a sorted-looking structure is automatically binary searchable — if the actual property you need to check isn't monotonic across the structure (the true/false pattern flips multiple times), binary search will silently give wrong answers rather than failing loudly.
- Trying to force binary search onto data with no clean monotonic property at all, when a full scan (or a different technique entirely) is the only correct approach — recognizing the absence of this property is just as important as recognizing its presence.

## 7. C# EXAMPLES — Simple → Intermediate → Advanced

**SIMPLE** — searching a rotated sorted array, the exact diagram above:

```

int SearchRotated(int[] arr, int target) {
    int left = 0, right = arr.Length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;

        bool leftHalfIsSorted = arr[left] <= arr[mid]; // the predicate
        if (leftHalfIsSorted) {
            if (arr[left] <= target && target < arr[mid]) right = mid - 1;
            else left = mid + 1;
        } else {
            if (arr[mid] < target && target <= arr[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}

```

At every step, the predicate “is the left half internally sorted?” tells you which half to trust for a direct range comparison, and which half to instead treat as “everything else” — the array isn't globally sorted, but this localized monotonic property is enough to drive correct elimination.

**INTERMEDIATE — finding the minimum in a rotated sorted array (finding the rotation point itself):**

```

int FindMin(int[] arr) {
    int left = 0, right = arr.Length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] > arr[right]) {
            left = mid + 1; // minimum is in the right half
        } else {
            right = mid; // minimum is at mid or in left half
        }
    }
    return arr[left];
}

```

The predicate here is “is arr[mid] > arr[right]?” — true means the rotation point (and thus the minimum) is still ahead in the right half; false means mid itself could be the minimum or it's somewhere in the left half. Note right = mid (not mid - 1) here, since mid itself remains a valid candidate — a direct preview of 6.4's off-by-one care.

**ADVANCED — “first bad version” style problem: finding the exact boundary where an externally-defined check first becomes true, without any array at all:**

```

// IsBadVersion(int version) is a given black-box predicate -- monotonic:
// false, false, ..., false, true, true, ..., true
int FirstBadVersion(int n) {
    int left = 1, right = n;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (IsBadVersion(mid)) {
            right = mid; // this version (or earlier) is bad
        } else {
            left = mid + 1; // this version is fine, bad is later
        }
    }
    return left; // left == right -- the exact boundary
}

```



There's no array here at all — `IsBadVersion` is an opaque function, and the entire search operates purely over the range of integers 1 to  $n$ , using the monotonic true/false pattern directly as the thing being binary searched, which is the clearest possible illustration of this topic's core point: binary search doesn't need a sorted array, it needs a monotonic predicate.

## 8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, this generalization is often described as “binary searching on a predicate” or “binary searching the answer space,” and it's the conceptual bridge directly into 6.3's “binary search on answer” pattern — the `FirstBadVersion` example is already binary search on answer, just without yet framing the search space as representing candidate solutions to an optimization problem. There's also a useful mental discipline worth adopting: explicitly write out, in words, exactly what your predicate function checks and confirm to yourself it's monotonic before writing the binary search loop — skipping this verification step is the most common reason people apply binary search incorrectly to problems that only superficially resemble searchable ones.

## 9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming “the data involves some sorted structure” is sufficient justification for binary search, without explicitly verifying that the specific property being checked is monotonic across that structure — a rotated array is still partially sorted, but naively applying plain binary search (6.1's version, unmodified) to it without accounting for the rotation produces wrong answers. Another is constructing a predicate that seems reasonable but actually isn't monotonic on certain edge cases (e.g., arrays with duplicate values complicating a rotation-point search) — verifying monotonicity needs to account for edge cases, not just the “clean” example used to first understand the pattern. A third: not recognizing binary-searchable problems when they're phrased as black-box function calls or abstract conditions rather than literal array lookups, as in the “first bad version” example — defaulting to a linear scan because no array is explicitly present, missing that the same  $O(\log n)$  technique still applies.

## 10. SELF-CHECK

- What is the actual, general requirement for a problem to be binary searchable — state it without using the word “sorted.”
- In the rotated-array search example, why does checking whether the left half is internally sorted let you decide which half to search next, even though the whole array isn't sorted?
- In the “first bad version” example, why does `right = mid` (not `mid - 1`) need to be used when the predicate is true, mirroring the same care needed in the `FindMin` example?

## 6.3 — Binary search on answer (search space reduction pattern)

### 1. PLAIN-LANGUAGE GIST

“Binary search on answer” means applying binary search not to positions in an array, but to the space of possible answers to an optimization problem — guessing a candidate answer, checking (via some feasibility test) whether that candidate works, and using the result to halve the remaining range of plausible answers, exactly like 6.2 generalized but now explicitly framed around solving “what’s the best/smallest/largest value satisfying some condition.”

### 2. REAL-WORLD ANALOGY

Picture trying to find the minimum amount of weight a moving truck needs to be able to carry per trip in order to move all your boxes within a certain number of trips. Rather than trying every possible truck capacity one by one starting from the smallest, you guess a plausible capacity, simulate how many trips that capacity would actually take, and use that result (too many trips means the guess was too small; an acceptable number of trips means the guess might be reducible) to narrow your guess — just like checking a midpoint and discarding half the range in classic binary search, except your “array” is the range of possible truck capacities.

### 3. CONNECT TO PRIOR KNOWLEDGE

This is the natural conclusion of 6.2’s generalization — 6.2 established that any monotonic predicate can be binary searched; this topic specifically applies that idea to optimization problems where the predicate is “is this candidate answer feasible/sufficient?” and the search space is a numeric range of possible answers, not array indices at all.

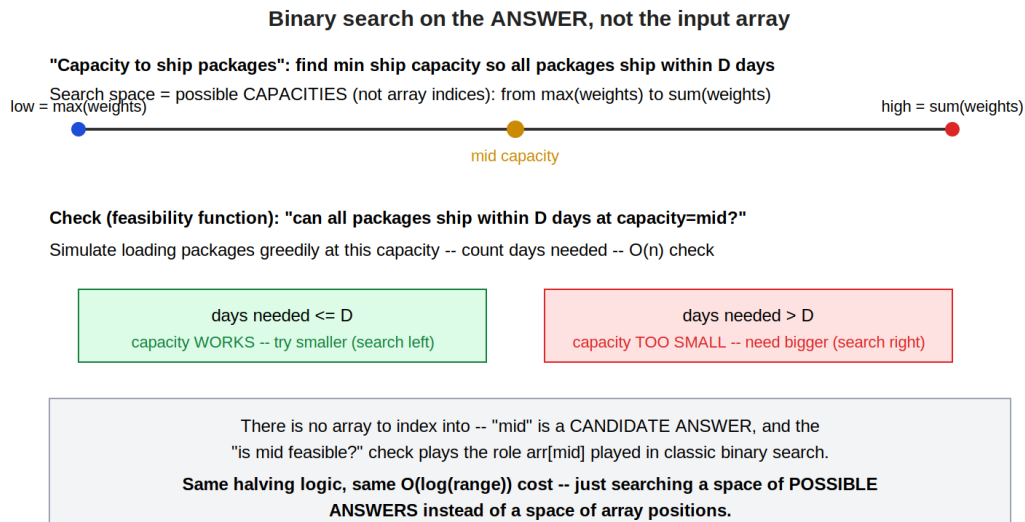
### 4. CORE MECHANISM (Simple → Intermediate)

**Simple model:** Identify the range of all possible answers (e.g., from the smallest conceivable value to the largest conceivable value). Binary search this range: for each candidate midpoint, run a feasibility check — usually an  $O(n)$  simulation or calculation — that tells you whether this candidate answer works. If it works, the true answer might be smaller (or larger, depending on the problem), so narrow accordingly; if it doesn’t work, push the search in the other direction.

**Next layer — recognizing the shape of these problems, and the cost structure:**

- The telltale phrasing: “find the minimum/maximum  $X$  such that  $Y$  is achievable,” “find the smallest capacity/value/threshold that satisfies a constraint” — these are strong signals that the answer itself (not array positions) should be the thing being binary searched.
- The feasibility check is usually the most involved part of the solution — often an  $O(n)$  greedy simulation, as in the “ship within  $D$  days” example previewed in 6.2’s diagrams — and the overall complexity becomes  $O(n \log(\text{range}))$ :  $O(\log(\text{range}))$  iterations of binary search, each costing  $O(n)$  to run the feasibility check, a direct combination of 6.1’s halving logic with a per-step linear cost, rather than 6.1’s  $O(1)$  per-step array comparison.
- Defining the correct initial low and high bounds for the answer range matters — too narrow risks excluding the true answer; too wide just costs a few extra (still logarithmic) iterations, so when in doubt, err toward a wider, clearly-safe initial range.

### 5. VISUAL REPRESENTATION



*The diagram shows the search space as a range of possible capacities (not array indices), with a feasibility check at each midpoint determining whether to search the smaller-capacity half or the larger-capacity half — directly extending 6.1's elimination logic onto a different kind of space entirely.*

## 6. WHEN TO USE / WHEN NOT TO USE

### Use binary search on answer when:

- The problem asks for an optimal threshold/capacity/value, and you can write a feasibility check — "does this candidate value satisfy the constraint?" — that's monotonic (smaller/larger candidates are progressively easier/harder to satisfy, with a clean transition point).
- A brute-force approach would be "try every possible answer from smallest to largest until one works," and you notice the feasibility check's result is monotonic across that range, just like 6.2's general recognition test.

### Wrong choice when:

- The feasibility check itself isn't monotonic across the candidate range — if some smaller answers work, some larger ones also work, but a few in between don't, binary search would incorrectly skip over valid regions; always verify monotonicity before committing to this approach.
- The problem doesn't actually have a numeric range of candidate answers to search — if the "answer" is more structural (an arrangement, a set, a path) rather than a single quantity sliding along a range, this pattern doesn't apply directly.

## 7. C# EXAMPLES — Simple → Intermediate → Advanced

**SIMPLE** — minimum capacity to ship packages within D days, the exact diagram above:

```

bool CanShipWithinDays(int[] weights, int capacity, int days) {
    int daysNeeded = 1, currentLoad = 0;
    foreach (int w in weights) {
        if (currentLoad + w > capacity) {
            daysNeeded++;
            currentLoad = 0;
        }
        currentLoad += w;
    }
    return daysNeeded <= days;
}

int MinCapacity(int[] weights, int days) {
    int low = weights.Max(), high = weights.Sum(); // safe bounds
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (CanShipWithinDays(weights, mid, days)) {
            high = mid; // mid works -- try smaller
        } else {
            low = mid + 1; // mid too small -- need bigger
        }
    }
    return low;
}

```

low starts at `weights.Max()` because no capacity smaller than the single heaviest package could ever work; high starts at the full sum because that capacity always works in exactly one day — both are provably safe starting bounds, not arbitrary guesses.

**INTERMEDIATE — minimum “speed” to eat all bananas within H hours (a very similarly-shaped problem, reinforcing the template):**

```

bool CanFinishAtSpeed(int[] piles, int speed, int hours) {
    int hoursNeeded = 0;
    foreach (int pile in piles) {
        hoursNeeded += (pile + speed - 1) / speed; // ceiling division
    }
    return hoursNeeded <= hours;
}

int MinEatingSpeed(int[] piles, int hours) {
    int low = 1, high = piles.Max();
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (CanFinishAtSpeed(piles, mid, hours)) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }
    return low;
}

```

Despite describing a completely different real-world scenario (eating bananas vs. shipping packages), the shape is identical to the simple example: define bounds, write a feasibility check, binary search for the boundary — reinforcing that recognizing this shared template matters more than memorizing any one specific problem.

**ADVANCED — binary searching a maximum feasible value (maximizing the minimum distance between aggressive cows in stalls) — illustrating the pattern's extension to searching for a MAXIMUM:**

```

bool CanPlaceCows(int[] stalls, int minDistance, int cows) {
    int placed = 1, lastPosition = stalls[0];
    for (int i = 1; i < stalls.Length; i++) {
        if (stalls[i] - lastPosition >= minDistance) {
            placed++;
            lastPosition = stalls[i];
        }
    }
    return placed >= cows;
}

int MaxMinDistance(int[] stalls, int cows) {
    Array.Sort(stalls); // sorting required for the greedy check
    int low = 0, high = stalls[^1] - stalls[0];
    while (low < high) {
        int mid = low + (high - low + 1) / 2; // NOTE: +1, searching MAX
        if (CanPlaceCows(stalls, mid, cows)) {
            low = mid; // mid works -- push distance LARGER
        } else {
            high = mid - 1;
        }
    }
    return low;
}

```

This variant searches for a maximum feasible value rather than a minimum, which flips which half gets kept on a successful check ( $\text{low} = \text{mid}$  instead of  $\text{high} = \text{mid}$ ) — and that flip requires the  $+1$  in the midpoint calculation to avoid an infinite loop when  $\text{low}$  and  $\text{high}$  are adjacent, a subtlety that's easy to get backwards and is examined directly in 6.4.

## 8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, binary search on answer is a specific instance of a broader optimization strategy sometimes called “parametric search” — converting an optimization problem (“find the best  $X$ ”) into a series of decision problems (“is  $X$  feasible?”), which is useful whenever the decision version is significantly easier to solve than the optimization version directly. There's also a recurring subtlety worth flagging explicitly: whether you're searching for a minimum feasible answer or a maximum feasible answer changes which direction  $\text{low}/\text{high}$  get updated on a successful check, and getting this backwards is a common, easy-to-make logic error — always sanity-check with a tiny example by hand (0.8's skill) before trusting the direction of narrowing.

## 9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is forgetting to verify that the feasibility check is actually monotonic across the candidate answer range before applying this pattern — assuming “it's a min/max problem, so binary search on answer must apply” without checking that smaller-vs-larger candidates behave consistently. Another is mixing up the update rules for “searching for minimum” versus “searching for maximum” — using  $\text{high} = \text{mid}$  when you actually need  $\text{low} = \text{mid}$  (or the reverse) silently searches in the wrong direction, converging to an incorrect boundary rather than crashing. A third: setting initial  $\text{low}/\text{high}$  bounds without proving they're actually safe (provably feasible/infeasible at the extremes) — arbitrary or unjustified bounds risk excluding the true answer from the search range entirely, a subtle bug that only manifests on specific inputs near the boundary you incorrectly excluded.

## 10. SELF-CHECK

- What's the general signal in a problem's wording that suggests “binary search on answer” rather than classic array-based binary search (6.1)?

- Why does the overall complexity of binary search on answer typically become  $O(n \log(\text{range}))$  rather than just  $O(\log(\text{range}))$  — what's contributing the extra  $O(n)$  factor?
- In the “maximum minimum distance” cow-placement example, why does searching for a maximum feasible value require  $\text{low} = \text{mid}$  on success (instead of  $\text{high} = \text{mid}$ ), and why does that change require the  $+1$  adjustment in the midpoint calculation?

## 6.4 — Common off-by-one mistakes and how to avoid them

### 1. PLAIN-LANGUAGE GIST

Binary search has a deceptively small amount of code, but a notoriously high rate of subtle bugs — almost all of them boundary/off-by-one errors involving the loop condition, the midpoint update rule, or the bounds themselves — and this topic is specifically about naming the recurring mistake patterns so you can check for them deliberately rather than debugging them by trial and error.

### 2. REAL-WORLD ANALOGY

Skipped — this topic is a direct, practical debugging checklist built from the mechanics already covered in 6.1–6.3, not a new concept needing a fresh metaphor. The value here is in the specific named traps, covered concretely in section 4 and the diagram.

### 3. CONNECT TO PRIOR KNOWLEDGE

This topic directly names and systematizes the subtle details already flagged in passing throughout 6.1–6.3 — the  $\text{right} = \text{mid}$  vs.  $\text{right} = \text{mid} - 1$  distinction in 6.2's FindMin, and the  $\text{low} = \text{mid}$  with a  $+1$  midpoint adjustment in 6.3's maximum-distance example — collecting them into one deliberate review checklist rather than leaving them as scattered footnotes.

### 4. CORE MECHANISM (Simple → Intermediate)

**Simple model:** There are three specific, recurring sources of binary search bugs, and each has a clear, checkable rule:

- **Loop condition mismatch:**  $\text{while}(\text{left} \leq \text{right})$  and  $\text{while}(\text{left} < \text{right})$  are both valid, but they demand different surrounding logic — mixing the wrong update rule with the wrong loop condition is the most common single mistake category.
- **Midpoint update forgetting to exclude mid:** writing  $\text{left} = \text{mid}$  (instead of  $\text{mid} + 1$ ) when you've just determined mid is not the answer leaves mid still inside the range, and since mid is computed from left/right via integer division, this can cause mid to stop changing entirely — an infinite loop, not just a slow one.
- **Integer overflow in midpoint calculation:**  $(\text{left} + \text{right}) / 2$  can overflow if  $\text{left} + \text{right}$  exceeds the maximum representable integer value;  $\text{left} + (\text{right} - \text{left}) / 2$  avoids this by never summing two potentially-large bounds directly.

**Next layer — the deeper pattern underneath all three:**

- Conditions 1 and 2 are really the same underlying issue from two different angles: every loop condition implicitly defines what “the search is over” means, and every update rule must be written to guarantee progress toward that exact stopping condition — when these two pieces don't agree (e.g., a loop condition that expects the range to shrink to a single element, paired with an update rule that allows the range to stay the same size), the algorithm either loops forever or terminates one step too early/late.
- A reliable debugging discipline: before trusting any binary search implementation, trace it by hand on a 2-3 element array (0.8's skill, directly applicable here) — specifically check what happens when left and right are adjacent or equal, since that's exactly where off-by-one bugs surface, and a binary search that “looks correct” on a 10-element array can still loop forever or terminate incorrectly on a 2-element edge case.

### 5. VISUAL REPRESENTATION

## Off-by-one traps: loop bound and search-direction boundaries

### Trap 1: while (left < right) vs while (left <= right)

left <= right  
stops when left > right (range is empty)

left < right  
stops when left == right (one element left)

Mixing these with the WRONG mid-update rule causes infinite loops or missed elements --  
each loop condition demands a specific, matching update rule below it.

### Trap 2: forgetting +1/-1 when narrowing, causing infinite loop

if (arr[mid] < target)  
left = mid; // BUG  
mid can equal left forever -- never shrinks

if (arr[mid] < target)  
left = mid + 1; // correct  
always excludes mid -- guaranteed to shrink

### Trap 3: mid = (left + right) / 2 overflow vs mid = left + (right-left)/2

left + (right-left)/2 avoids integer overflow when left+right exceeds int range --  
rarely triggered in small interview inputs, but the "always write it this way" habit costs nothing.

*The diagram contrasts the two valid loop-condition styles and their required matching update rules side by side, then shows the specific "forgot to exclude mid" bug that causes an infinite loop, and finally the integer-overflow-safe midpoint formula — three distinct, nameable traps shown concretely rather than abstractly.*

## 6. WHEN TO USE / WHEN NOT TO USE

### Apply this checklist when:

- You've just written any binary search variant (classic, boundary-finding, on-answer) and want to verify correctness before trusting it, especially under interview time pressure where a quick, deliberate check beats an anxious re-read of the same code.
- A binary search implementation is producing an infinite loop or an off-by-one wrong answer, and you need a structured way to localize the bug rather than guessing randomly at fixes.

This isn't really an optional "when to use" — it's a verification habit worth applying every single time, given how often even experienced engineers introduce exactly these three bugs.

## 7. C# EXAMPLES — Simple → Intermediate → Advanced

### SIMPLE — the mismatched loop condition / update rule bug, shown explicitly:

```
// BUGGY: while (left <= right) paired with right = mid (not mid - 1)
int BuggySearch(int[] arr, int target) {
    int left = 0, right = arr.Length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid;           // BUG: should be mid - 1
    }
    return -1;
}
// When arr[mid] > target and mid == right, "right = mid" doesn't
// shrink the range -- left <= right stays true forever -- infinite loop.
```

### INTERMEDIATE — the corrected version, with the matching, consistent rule pair:



```

int FixedSearch(int[] arr, int target) {
    int left = 0, right = arr.Length - 1;
    while (left <= right) { // loop condition: "<="
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1; // always excludes mid
        else right = mid - 1; // always excludes mid
    }
    return -1;
}
// Both update rules exclude mid, guaranteeing the range strictly
// shrinks every iteration -- consistent with "<=" loop assumption.

```

**ADVANCED — verifying a boundary-search implementation by hand-tracing the smallest possible non-trivial case (applying 0.8's skill directly to this topic):**

```

int LeftBoundary(int[] arr, int target) {
    int left = 0, right = arr.Length; // note: right starts OUT of bounds
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] < target) left = mid + 1;
        else right = mid; // mid might BE the boundary
    }
    return left;
}
// Hand-trace on arr = [2, 2], target = 2:
// left=0, right=2, mid=1 -> arr[1]=2, not < target -> right = mid = 1
// left=0, right=1, mid=0 -> arr[0]=2, not < target -> right = mid = 0
// left=0, right=0 -> loop ends (left < right is false)
// returns 0 -- correctly finds the LEFTMOST occurrence

```

This deliberately uses the `left < right` loop condition paired with `right = mid` (not `mid - 1`) — correct here specifically because `mid` itself might be the answer being searched for (the leftmost boundary), unlike the classic exact-match search where a non-matching `mid` must always be excluded. Recognizing which style a given binary search variant needs — and verifying it by hand-tracing the smallest meaningful case — is the practical skill this entire topic builds toward.

## 8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, there's a well-known unifying framework for binary search variants (sometimes taught as “binary search templates”) that explicitly categorizes problems by exactly which loop condition and update-rule pairing they need, based on whether `mid` itself can ever be a valid answer (boundary-finding problems, which keep `mid` in play via `right = mid`) versus problems where a non-matching `mid` must always be discarded (exact-match search, which always excludes `mid` via `mid ± 1`). Internalizing this small set of named templates, rather than re-deriving the correct update rule from scratch under pressure each time, is part of what makes binary search fast and reliable to produce correctly in an interview setting once it's been deliberately practiced this way.

## 9. COMMON MISCONCEPTIONS AND MISTAKES

The single most common mistake, underlying nearly every binary search bug, is using an update rule that doesn't strictly exclude `mid` when `mid` has been determined not to be the answer — this is what causes the classic infinite loop, and it's worth specifically checking for on every binary search you write, not just debugging reactively when a loop hangs. Another is assuming all binary search variants use the same update rule template — boundary-finding searches (like `LeftBoundary` above) legitimately need `right = mid` (keeping `mid` as a candidate), which looks suspiciously similar to the classic infinite-loop bug but is actually correct in that specific context, making it easy to “fix” a correct implementation into a broken one by applying the wrong lesson

from a different binary search variant. A third: skipping the hand-trace verification step on a 2-3 element edge case, relying instead on the code “looking right” — many off-by-one bugs are entirely invisible on larger test inputs and only surface at the small boundary cases that a quick by-hand trace would catch immediately.

## 10. SELF-CHECK

- Walk through, out loud, exactly why while ( $\text{left} \leq \text{right}$ ) paired with  $\text{right} = \text{mid}$  (instead of  $\text{mid} - 1$ ) causes an infinite loop — at what specific point does the range stop shrinking?
- In the LeftBoundary example, why is  $\text{right} = \text{mid}$  actually correct here, even though it looks like the same pattern flagged as a bug in the simple example?
- Why is hand-tracing a binary search on a tiny 2-3 element array specifically useful for catching off-by-one bugs that might not appear on larger test inputs?